

# **Использованием методов каузального вывода при анализе зависимостей между оптимизирующими проходами LLVM-компилятора**

Выполнил: Белонин Владимир Викторович (М05-002д)

Научный руководитель: Ефанов Николай Николаевич, к.ф.-м.н., доцент

Московский физико-технический институт (МФТИ)

2026

# Проблема оптимизации компилятора

---

## Масштаб проблемы

- Современные компиляторы содержат **150–200+ проходов** оптимизации (GCC, LLVM)
- Стандартные уровни ( -01 , -02 , -03 ) — фиксированные последовательности
- Они оптимальны *в среднем*, но не для конкретной программы

## Комбинаторный взрыв ( $n = 50$ проходов)

Выбор проходов:  $2^n$  комбинаций

при  $n=50 \rightarrow \sim 10^{15}$  вариантов

Упорядочивание:  $n!$  перестановок

при  $n=50 \rightarrow \sim 10^{64}$  вариантов

**Ключевая сложность:** проходы не независимы — они взаимодействуют друг с другом. Один проход может создавать или уничтожать возможности для другого.



# Обзор существующих подходов

## Wang & O'Boyle (2018)

*"Machine Learning in Compiler Optimisation"*

- Систематизация ML-подходов: feature engineering → обучение модели → deployment
- Классификация методов: supervised (регрессия, классификация), unsupervised (кластеризация), reinforcement learning

## Ashouri et al. (2018)

*"A Survey on Compiler Autotuning using Machine Learning"*

- Формализация двух проблем: *selection problem* ( $2^n$ ) и *phase-ordering problem* ( $n!$ )
- Обзор 200+ работ: итеративная компиляция, генетические алгоритмы, байесовские сети

## Ограничения существующих подходов

| Метод                  | Что делает                        | Что не объясняет         |
|------------------------|-----------------------------------|--------------------------|
| Итеративная компиляция | Находит хорошие комбинации        | Почему они работают      |
| ML-предсказание        | Корреляции features → performance | Каузальные механизмы     |
| Генетические алгоритмы | Эволюция последовательностей      | Структуру взаимодействий |

# Цель и задачи исследования

---

## Цель работы

Исследовать **каузальные зависимости** между оптимизирующими проходами LLVM и построить граф их взаимодействий для понимания структуры оптимизационного пространства.

## Задачи

1. Собрать датасет экспериментов с различными комбинациями проходов на стандартных бенчмарках
2. Применить методы каузального вывода для оценки эффектов отдельных проходов
3. Выявить *enabling*-зависимости (проход A создаёт возможности для B) и *disabling*-зависимости (проход A блокирует B)
4. Построить граф каузальных зависимостей между проходами
5. Валидировать результаты на отложенной выборке

# Формальная постановка задачи

## Определения

Пусть  $P = \{p_1, p_2, \dots, p_n\}$  — конечное множество оптимизирующих пассов компилятора.

Пусть  $\mathcal{X}$  — пространство программ,  $x \in \mathcal{X}$  — конкретная программа.

Вектор воздействий (treatment vector):

$$T = (T_1, T_2, \dots, T_n) \in \{0, 1\}^n, \text{ где } T_i = 1[\text{пасс } p_i \text{ применён}]$$

Потенциальные исходы (potential outcomes framework):

$$Y(t) : \{0,1\}^n \rightarrow \mathbb{R} \text{ — производительность программы при воздействии } t$$

## Целевые величины

| Величина                 | Определение                                                                                       |
|--------------------------|---------------------------------------------------------------------------------------------------|
| Average Treatment Effect | $\tau_i = \mathbb{E}_X[Y(T_i=1, T_{-i}) - Y(T_i=0, T_{-i})]$                                      |
| Conditional ATE          | $\tau_i(x) = \mathbb{E}[Y(T_i=1, T_{-i}) - Y(T_i=0, T_{-i}) \mid X = x]$                          |
| Interaction Effect       | $\gamma_{ij} = \mathbb{E}[Y(T_i=1, T_j=1) - Y(T_i=1, T_j=0) - Y(T_i=0, T_j=1) + Y(T_i=0, T_j=0)]$ |



## Задача

**Дано:** выборка  $\mathcal{D} = \{(x_k, T_k, Y_k)\}_{k=1}^K$ , где  $T_k \sim \text{Uniform}(\{0,1\}^n)$  — рандомизированные эксперименты.

**Найти:**

1. Оценки  $\hat{t}_i$  для всех  $p_i \in P$
2. Оценки  $\hat{y}_{i,j}$  для всех пар  $(p_i, p_j), i \neq j$
3. Граф зависимостей  $G = (P, E)$ , где  $E = \{(p_i, p_j) : |\hat{y}_{i,j}| > \theta\}$

**Обозначение:**  $T_{-i} = (T_1, \dots, T_{i-1}, T_{i+1}, \dots, T_n)$  — вектор воздействий без компоненты  $i$ .



# Почему каузальный вывод?

## Проблема корреляционного подхода

**ML-модель говорит:**  
«loop-unroll коррелирует с высоким speedup»

Но это может быть потому, что loop-unroll чаще применяют к программам с циклами, которые и так легче оптимизировать.

**Каузальная модель говорит:**  
«loop-unroll *вызывает* +5% speedup для программ с циклами»

Контролируем характеристики программы (конфаундеры) и измеряем истинный эффект.

## Преимущества каузального подхода

| Аспект         | ML (корреляции)                  | Causal Inference                      |
|----------------|----------------------------------|---------------------------------------|
| Вопрос         | Что обычно происходит вместе?    | Что произойдёт, если применить пасс?  |
| Интерпретация  | Чёрный ящик                      | Понятные причинно-следственные связи  |
| Обобщаемость   | Ломается при смене распределения | Каузальные связи устойчивы            |
| Взаимодействия | Сложно выявить                   | Явно моделируются через $\gamma_{ij}$ |

## Какие вопросы позволяет решить?

### ✓ Отбор пассив:

Какие пассивы имеют нулевой эффект и могут быть исключены?

### ✓ Порядок применения:

Если А создаёт возможности для В, то А должен идти раньше.

### ✓ Конфликты:

Какие пассивы не стоит применять вместе ( $\gamma < 0$ )?

### ✓ Синергии:

Какие пассивы усиливают друг друга ( $\gamma > 0$ )?



# Методология исследования

---



## Сбор данных

- ~20–30 ключевых проходов LLVM
- Рандомизированные комбинации
- ~5000–10000 экспериментов на бенчмарк
- Контроль: фиксированный seed, среднее по 5 запускам

## Метрики производительности

- **Время выполнения** (основная)
- Размер кода (code size)
- Время компиляции
- Инструкций на цикл (IPC)
- Cache misses (через perf)

## Методы каузального анализа

- **Causal Forests (econml)** — оценка гетерогенных эффектов CATE
- **DoWhy** — фреймворк для каузального вывода с refutation tests
- **Propensity Score Matching** — контроль confounders

# Ожидаемые результаты

---

## Планируемые выходные артефакты

### 1. Оценки каузальных эффектов

Количественная оценка влияния каждого прохода на производительность

### 3. Граф зависимостей

- Enabling-рёбра:  $A \rightarrow B$
- Disabling-рёбра:  $A \nrightarrow B$

### 2. Матрица взаимодействий

Синергия ( $\gamma > 0$ ) или антагонизм ( $\gamma < 0$ ) между парами проходов

### 4. Рекомендации

Порядок применения проходов на основе каузальных зависимостей

## Пример гипотезы для проверки

`mem2reg` → `instcombine` имеет положительный эффект взаимодействия (enabling), так как продвижение памяти в регистры создаёт паттерны для комбинирования инструкций.



# Заключение

---

## Основные тезисы

- Проблема оптимизации компилятора **актуальна** — стандартные уровни не оптимальны для конкретных программ
- Существующие ML-подходы находят корреляции, но не объясняют **причинно-следственные связи**
- Каузальный анализ позволяет понять **структуру зависимостей** между проходами
- Результаты могут быть использованы для построения **более эффективных эвристик**

## Текущий статус работы

✓ Изучена литература

✓ Выбрана методология

✓ Определены проходы для анализа

○ Сбор данных (в процессе)

**Спасибо за внимание!**

Готов ответить на вопросы